

Exp1: AIM: Linear Regression	EXP2: AIM: multivariate linear regression
<pre> import numpy as np from sklearn.model_selection import train_test_split from sklearn.linear_model import LinearRegression import matplotlib.pyplot as plt # Generate some example data # X is the independent variable, y is the dependent variable np.random.seed(0) X = 2 * np.random.rand(100, 1) y = 4 + 3 * X + np.random.randn(100, 1) # Split the data into training and testing sets X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) # Create the Linear Regression model model = LinearRegression() # Train the model using the training sets model.fit(X_train, y_train) # Make predictions using the testing set y_pred = model.predict(X_test) # Print coefficients print("Coefficients:\n", model.coef_) print("Intercept:\n", model.intercept_) # Plot outputs plt.scatter(X_test, y_test, color='black', label='Actual data') plt.plot(X_test, y_pred, color='blue', linewidth=3, label='Regression line') plt.xlabel("X") plt.ylabel("y") plt.title("Linear Regression") plt.legend() plt.show() # Evaluate the model print(f"Mean squared error: {np.mean((y_pred - y_test) ** 2)}") print(f"Coefficient of determination (R^2): {model.score(X_test, y_test)}") </pre>	<pre> import numpy as np from sklearn.model_selection import train_test_split from sklearn.linear_model import LinearRegression import matplotlib.pyplot as plt from mpl_toolkits.mplot3d import Axes3D # Generate some example data np.random.seed(0) X1 = 2 * np.random.rand(100, 1) X2 = 3 * np.random.rand(100, 1) y = 4 + 3 * X1 + 2 * X2 + np.random.randn(100, 1) # Combine X1 and X2 into a single matrix X = np.hstack((X1, X2)) # Split the data into training and testing sets X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) # Create the Linear Regression model model = LinearRegression() # Train the model using the training sets model.fit(X_train, y_train) # Make predictions using the testing set y_pred = model.predict(X_test) # Print coefficients print("Coefficients:\n", model.coef_) print("Intercept:\n", model.intercept_) # 3D Plot fig = plt.figure() ax = fig.add_subplot(111, projection='3d') ax.scatter(X_test[:, 0], X_test[:, 1], y_test, color='black', label='Actual data') ax.scatter(X_test[:, 0], X_test[:, 1], y_pred, color='blue', label='Predicted data') ax.set_xlabel("X1") ax.set_ylabel("X2") ax.set_zlabel("y") plt.title("Multivariate Linear Regression") plt.legend() plt.show() # Evaluate the model print(f"Mean squared error: {np.mean((y_pred - y_test) ** 2)}") print(f"Coefficient of determination (R^2): {model.score(X_test, y_test)}") </pre>

EXP:3:AIM: Logistic Regression	EXP:5:AIM: Support Vector Machine (SVM)
<pre> import numpy as np from sklearn.model_selection import train_test_split from sklearn.linear_model import LogisticRegression from sklearn.metrics import classification_report, confusion_matrix import matplotlib.pyplot as plt # Generate some example data np.random.seed(0) X = np.random.rand(100, 2) # 100 samples with 2 features y = (X[:, 0] + X[:, 1] > 1).astype(int) # Binary target variable # Split the data into training and testing sets X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) # Create the Logistic Regression model model = LogisticRegression() # Train the model using the training sets model.fit(X_train, y_train) # Make predictions using the testing set y_pred = model.predict(X_test) # Print evaluation metrics print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred)) print("\nClassification Report:\n", classification_report(y_test, y_pred)) # Optional: Visualize the decision boundary x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5 y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5 xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01), np.arange(y_min, y_max, 0.01)) Z = model.predict(np.c_[xx.ravel(), yy.ravel()]) Z = Z.reshape(xx.shape) plt.contourf(xx, yy, Z, alpha=0.8, cmap=plt.cm.Spectral) plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', marker='o', cmap=plt.cm.Spectral) plt.xlabel('Feature 1') plt.ylabel('Feature 2') plt.title('Logistic Regression Decision Boundary') plt.show() </pre>	<pre> # Load the important packages from sklearn.datasets import load_breast_cancer import matplotlib.pyplot as plt from sklearn.inspection import DecisionBoundaryDisplay from sklearn.svm import SVC # Load the dataset cancer = load_breast_cancer() X = cancer.data[:, :2] # Use only the first two features for visualization y = cancer.target # Build the model svm = SVC(kernel="rbf", gamma=0.5, C=1.0) # Train the model svm.fit(X, y) # Plot Decision Boundary DecisionBoundaryDisplay.from_estimator(svm, X, response_method="predict", cmap=plt.cm.Spectral, alpha=0.8, xlabel=cancer.feature_names[0], ylabel=cancer.feature_names[1],) # Scatter plot plt.scatter(X[:, 0], X[:, 1], c=y, s=20, edgecolors="k") plt.title("SVM Decision Boundary (RBF Kernel)") plt.show() </pre>

EXP:6:AIM: K-Nearest Neighbors (KNN)	EXP:7:AIM: DBSCAN clustering
<pre> # Import required libraries from sklearn.datasets import load_iris from sklearn.model_selection import train_test_split from sklearn.neighbors import KNeighborsClassifier from sklearn.metrics import accuracy_score, classification_report # Load the Iris dataset iris = load_iris() X = iris.data # Feature matrix y = iris.target # Labels # Split the dataset into training and testing sets (70% train, 30% test) X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42) # Initialize the KNN classifier with k=3 knn = KNeighborsClassifier(n_neighbors=3) # Train the KNN classifier knn.fit(X_train, y_train) # Predict on test data y_pred = knn.predict(X_test) # Evaluate the model print("Accuracy:", accuracy_score(y_test, y_pred)) print("\nClassification Report:\n", classification_report(y_test, y_pred, target_names=iris.target_names)) </pre>	<pre> import numpy as np import matplotlib.pyplot as plt from sklearn.datasets import make_blobs from sklearn.cluster import DBSCAN from sklearn.preprocessing import StandardScaler # Generate synthetic data n_samples = 750 X, _ = make_blobs(n_samples=n_samples, centers=4, cluster_std=0.60, random_state=0) # Standardize features X = StandardScaler().fit_transform(X) # Apply DBSCAN dbscan = DBSCAN(eps=0.3, min_samples=10) dbscan.fit(X) # Get cluster labels labels = dbscan.labels_ # Number of clusters (excluding noise) n_clusters = len(set(labels)) - (1 if -1 in labels else 0) print(f"Number of clusters found: {n_clusters}") print(f"Number of noise points: {list(labels).count(-1)}") # Plotting the results plt.figure(figsize=(10, 6)) # Plot each cluster unique_labels = set(labels) for label in unique_labels: if label == -1: # Noise points plt.scatter(X[labels == label, 0], X[labels == label, 1], s=50, c='black', marker='x', label='Noise') else: plt.scatter(X[labels == label, 0], X[labels == label, 1], s=50, label=f'Cluster {label}') plt.title('DBSCAN Clustering') plt.xlabel('Feature 1') plt.ylabel('Feature 2') plt.legend() plt.show() </pre>